

Doubly Linked List – Insert After Element And Time Complexity.

Program :

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted after " << element << "." <<
endl;
}

..... •
```

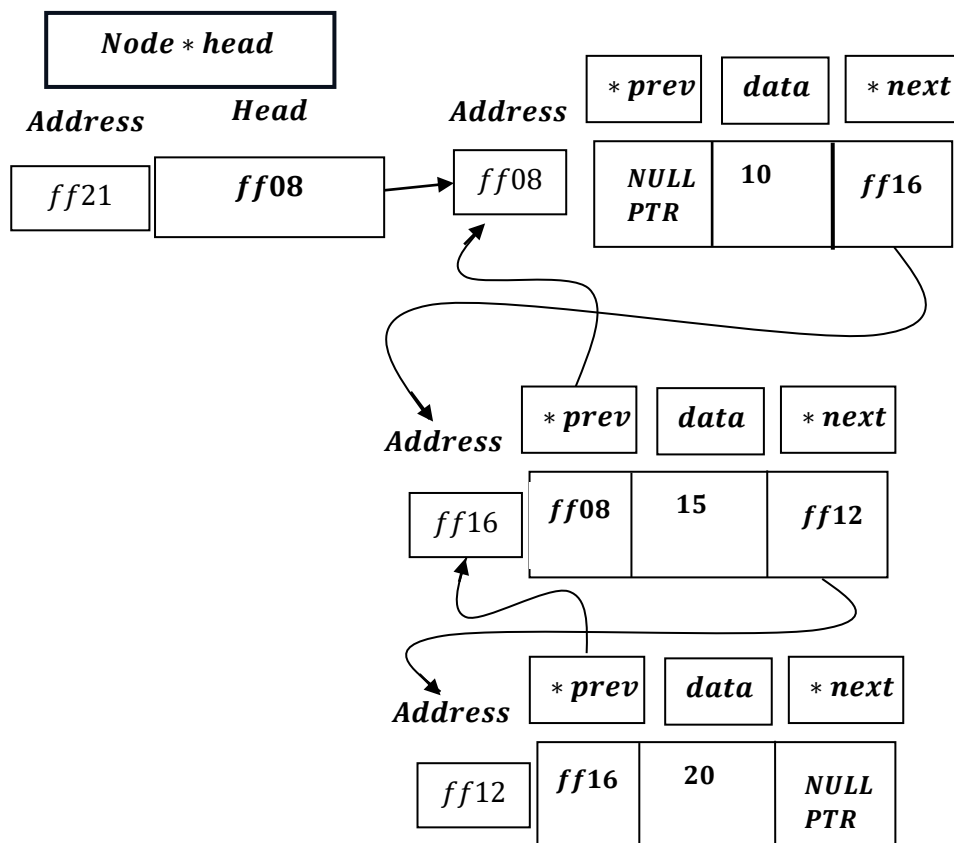
```

case 4:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter the element to insert after: ";
    cin >> element;
    insertAfterElement(data, element);
    break;

```

Program Analysis:

Suppose we have a list:



```

void insertAfterElement(int data, int element)
{.....}

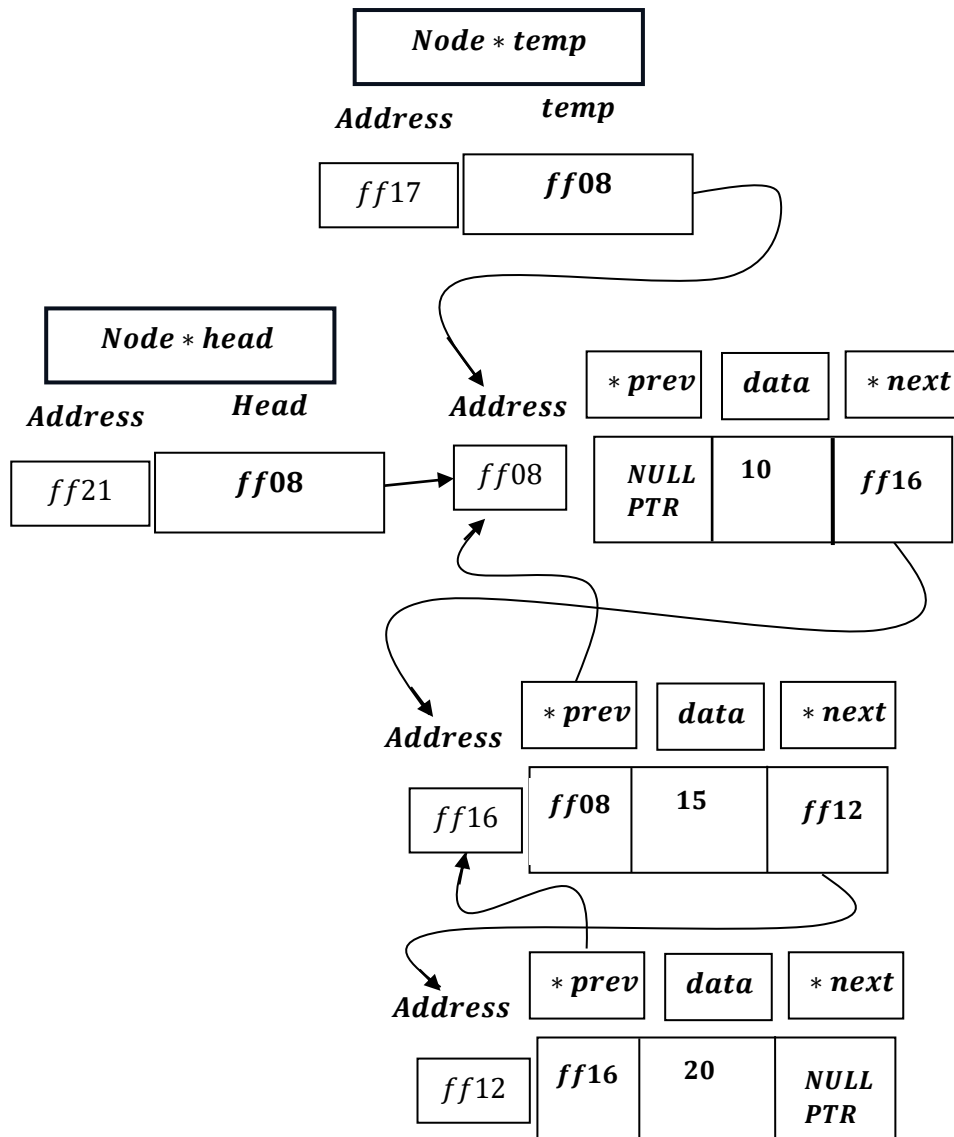
```

Here data → is for new user input.

element → The data is to be input , after the element , present in the list.

Next,

```
Node *temp = head;
```



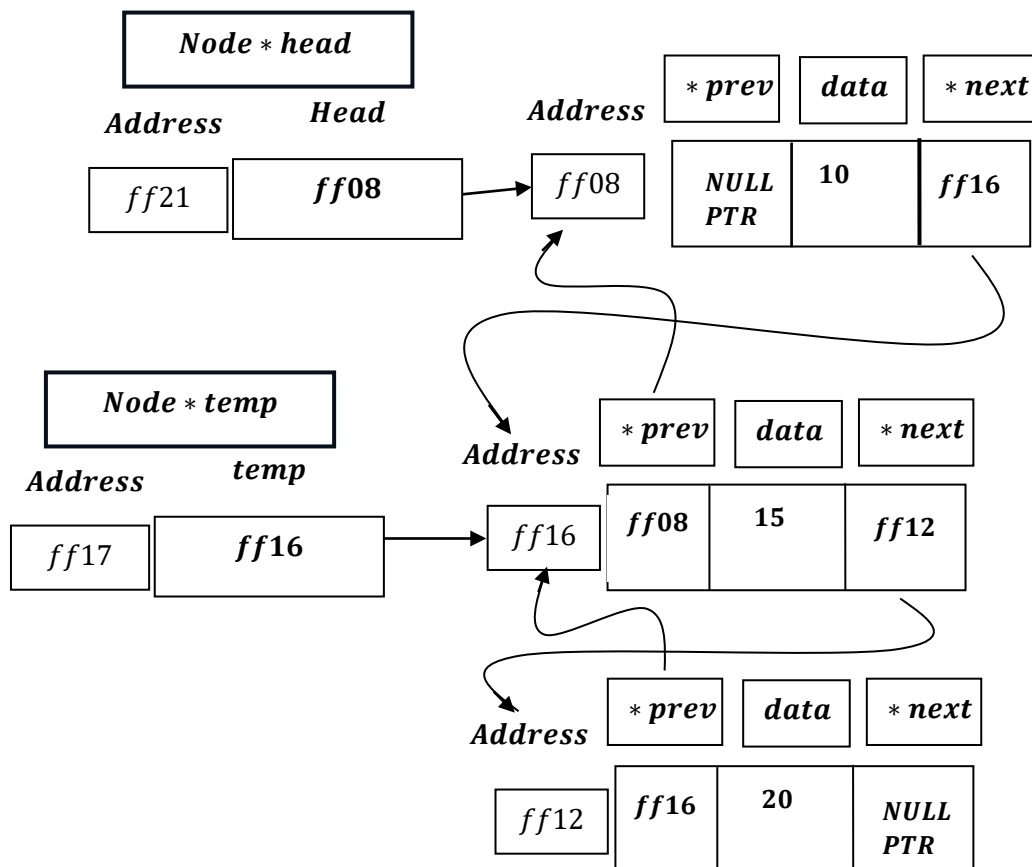
Lets input for the worst case, lets enter data = 25 after, Element = 20, hence condition is : $temp \rightarrow data \neq element$ i.e. $temp \rightarrow data \neq 20$, which is true and $temp \neq nullptr$ i.e. temp will point to the last node , i.e.

current n^{th} node.

```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

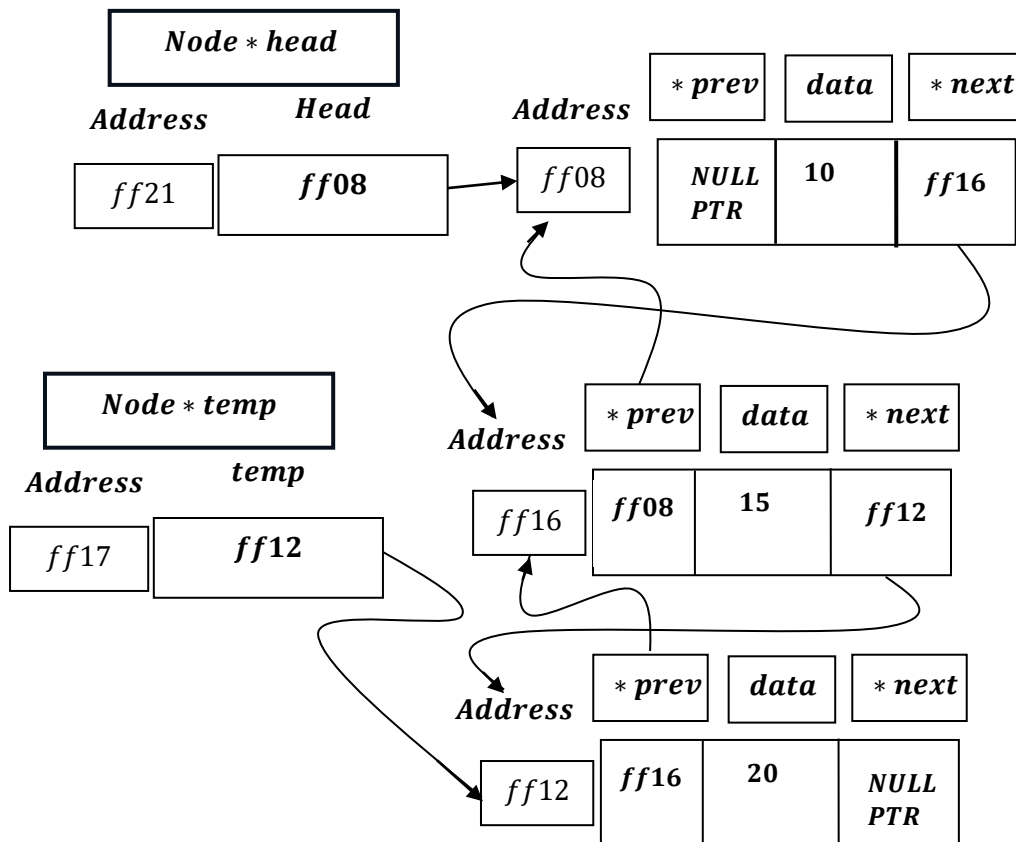
CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$

$TEMP \rightarrow DATA = 10 \neq ELEMENT = 20$, THEN , $TEMP = TEMP \rightarrow NEXT = ff16$.



CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$

$TEMP \rightarrow DATA = 15 \neq ELEMENT = 20$, THEN , $TEMP = TEMP \rightarrow NEXT = ff12$.



CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$
 $TEMP \rightarrow DATA = 20 \neq ELEMENT = 20$, Condition becomes false , hence ,
we get out of loop.

Next , it will check $temp = nullptr$, which is false , it exit if {...} block.

```
if (temp == nullptr){....}
```

As $temp = nullptr$, it will print , the user input Element is not found in the list.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a Node structure and *newNode* is declared as a pointer to Node structure.

→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

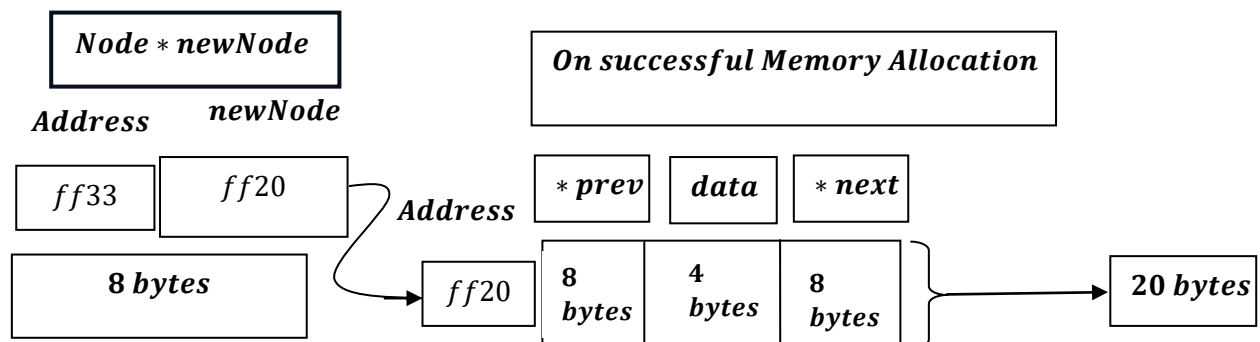
→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another Node) is typically 8 bytes.

→ *prev*(a pointer to previous Node) is typically 8 bytes.

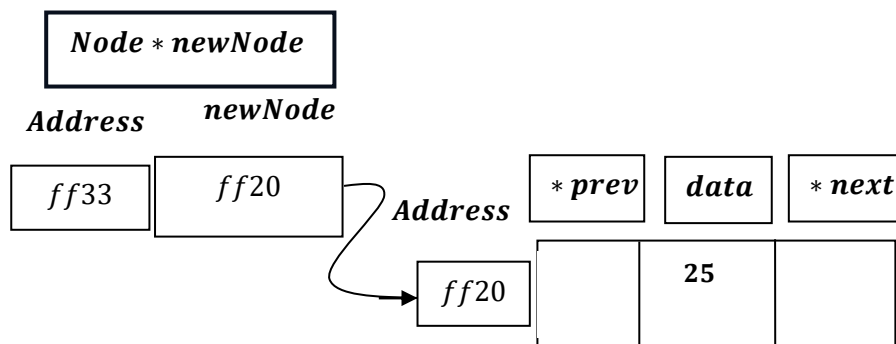
→ Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for previous) .

Say,



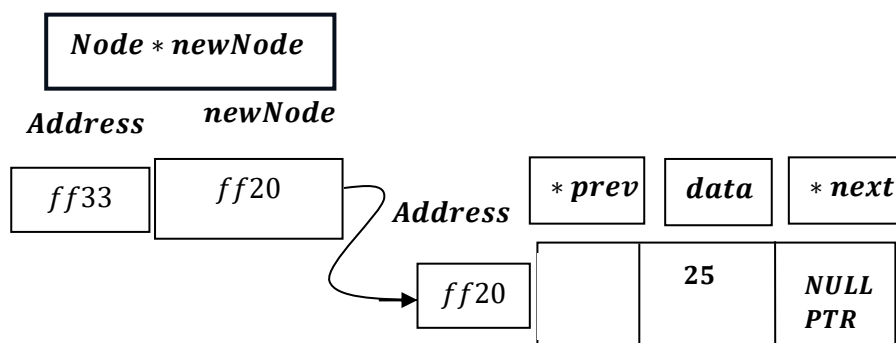
```
newNode->data = data;
```

newNode → *data* = 25,



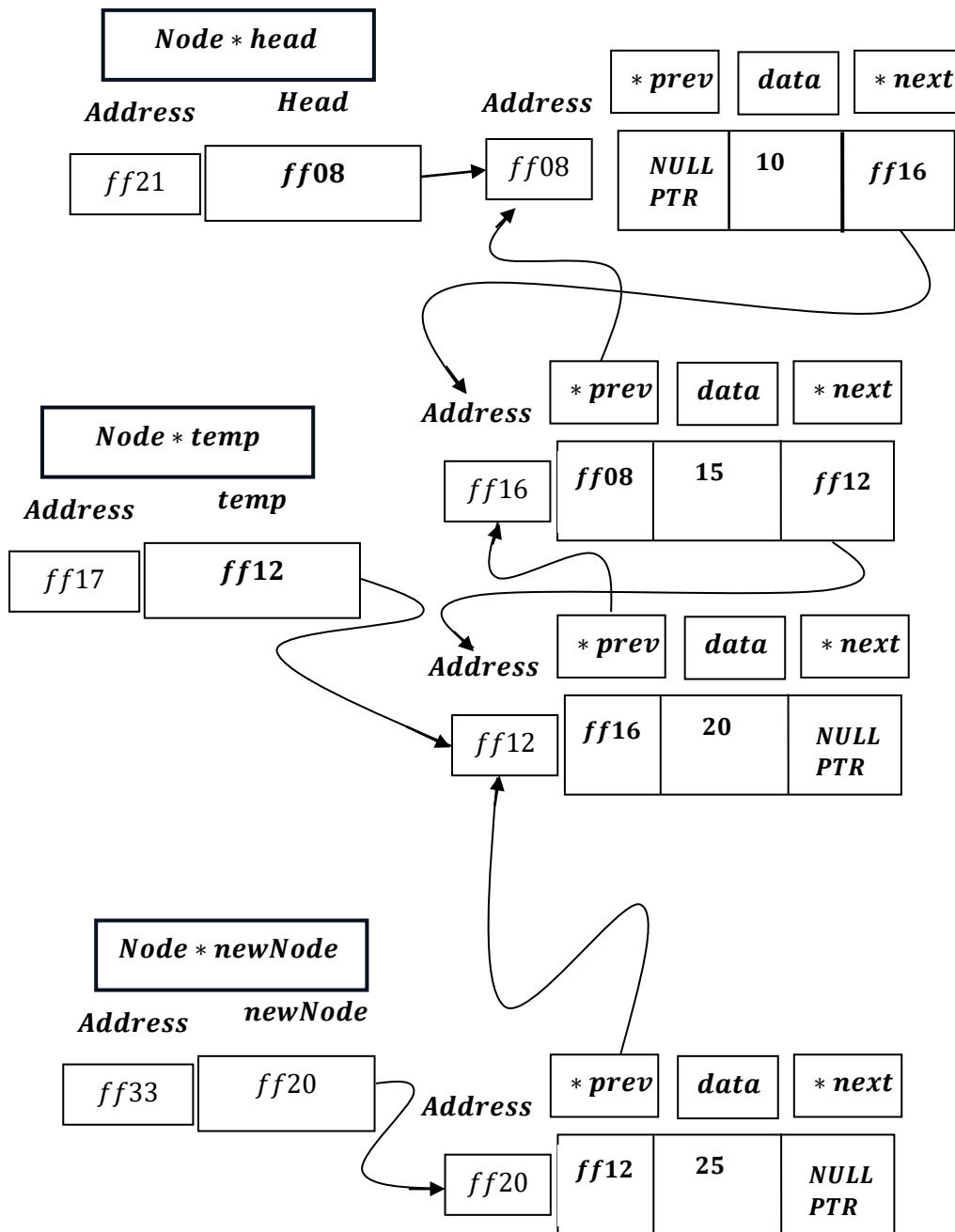
```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = nullptr.$



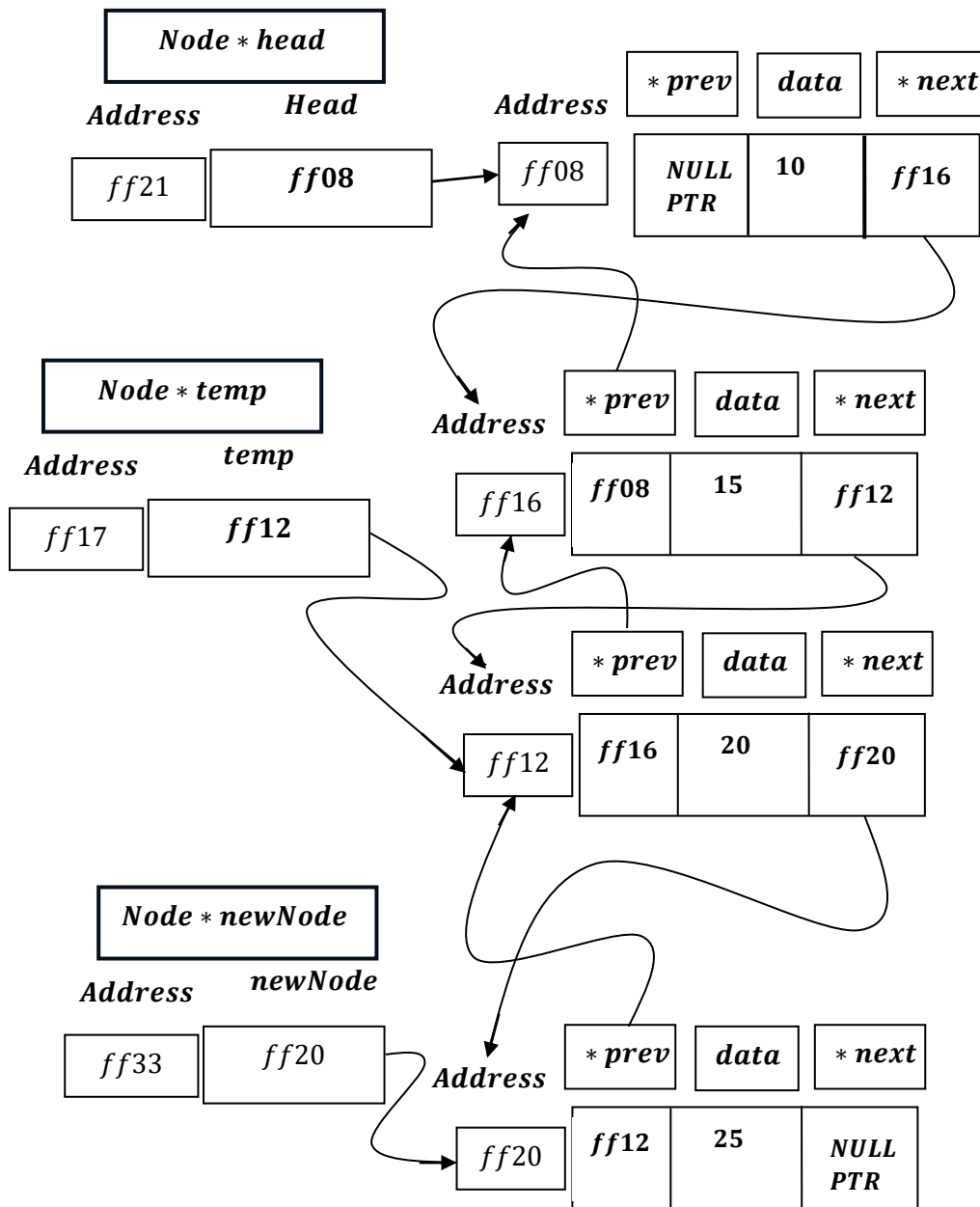
```
newNode->prev = temp;
```

$newNode \rightarrow prev = temp = ff12$

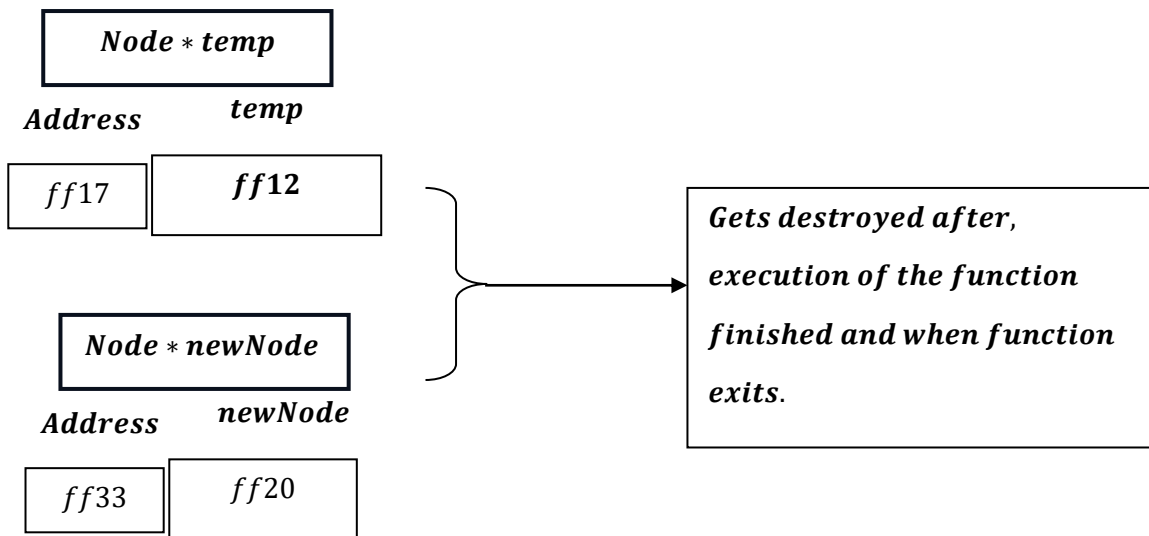


```
temp->next = newNode;
```

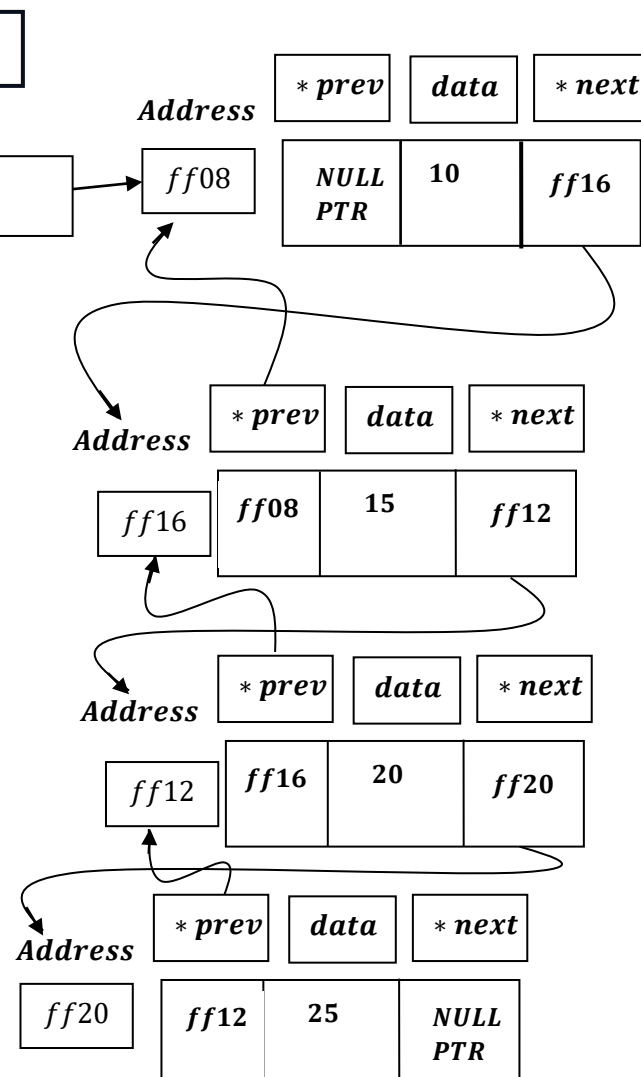
$temp \rightarrow next = newNode = ff20.$



Now, **Node * newNode** , **Node * temp** temporary variables, therefore gets destroyed after function execution get finished .



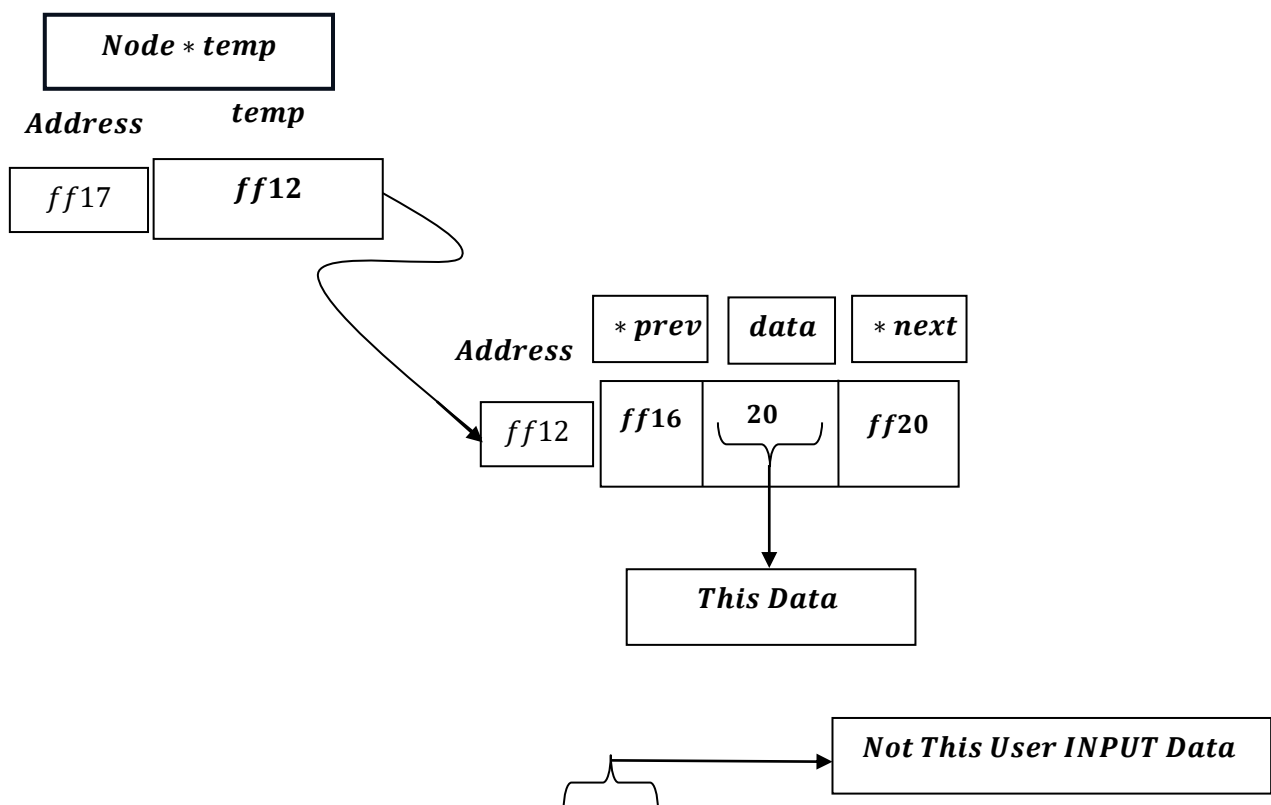
Hence ,we are left with :



Temp = NULLPTR or Temp ≠ NULLPTR and
TEMP → DATA = ELEMENT or TEMP → DATA ≠ ELEMENT

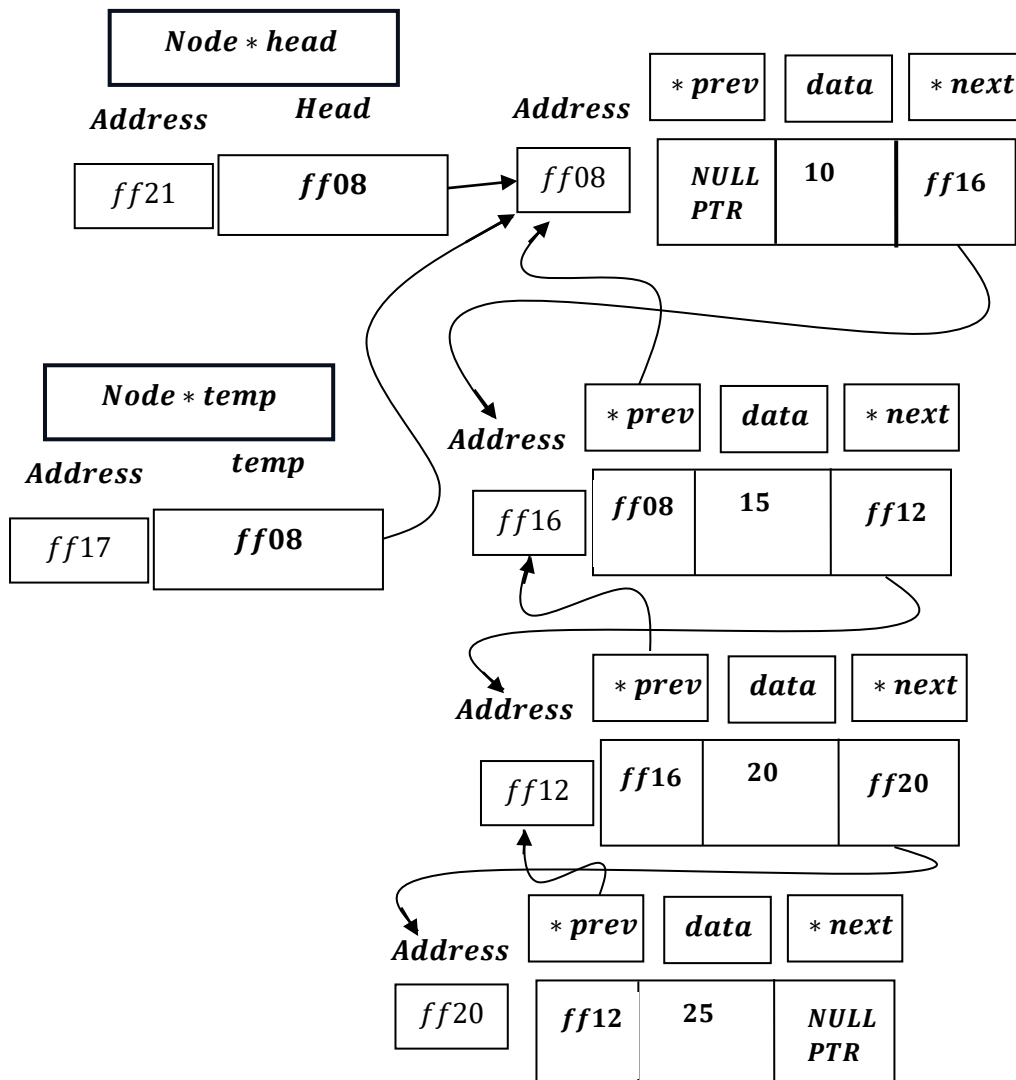
```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

TEMP → DATA is not the USER INPUT DATA but TEMP pointed DATA already stored in NODE.



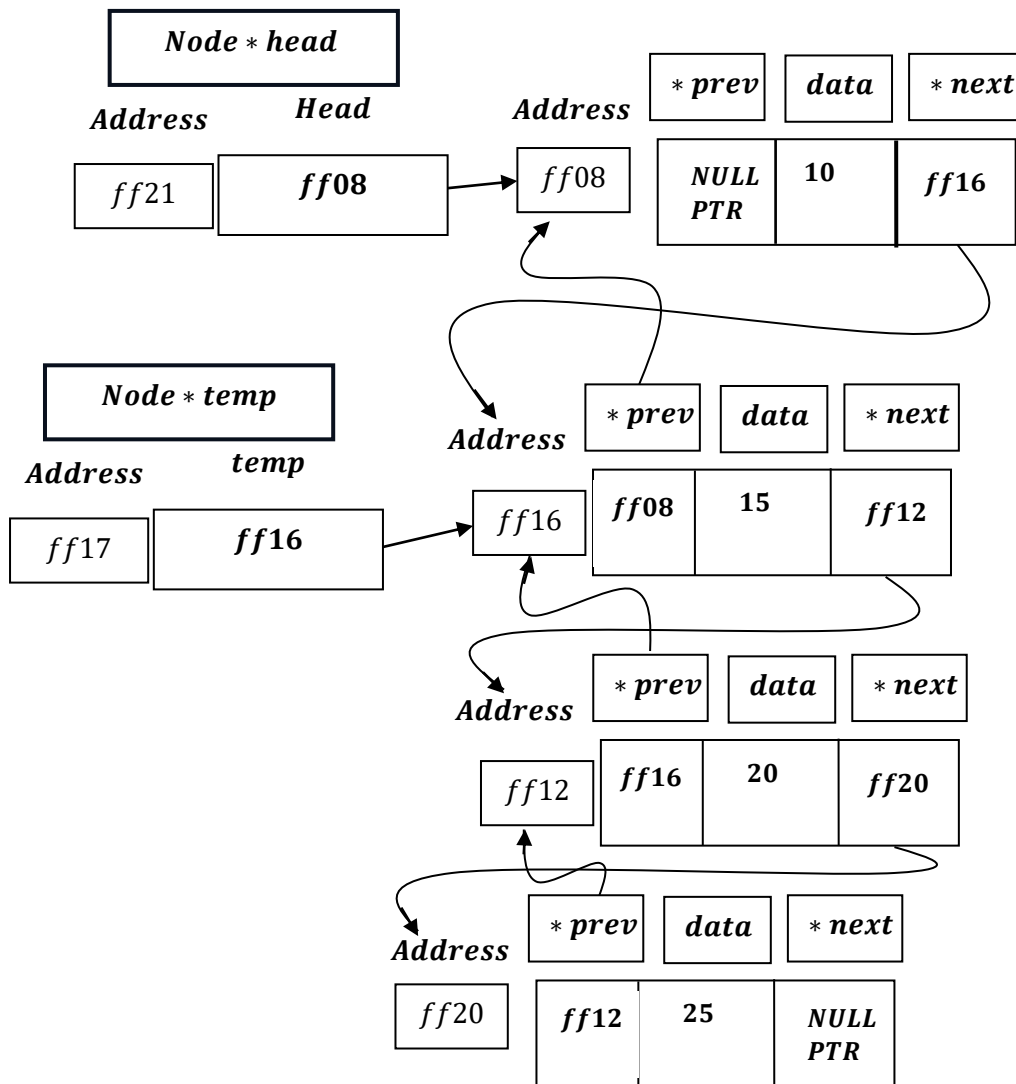
```
void insertAfterElement(int data, int element)
{.....}
```

Now say, we have a list:



And we want to enter data : 23 after element : 20.

*Now ,user input `element : 20` ,and $TEMP \rightarrow DATA \neq Element \neq 20$,hence ,
 $TEMP \rightarrow DATA$ will be 15 and $TEMP = TEMP \rightarrow NEXT = ff16$ and according to
condition of the loop: $TEMP \neq NULLPTR$ is true.*



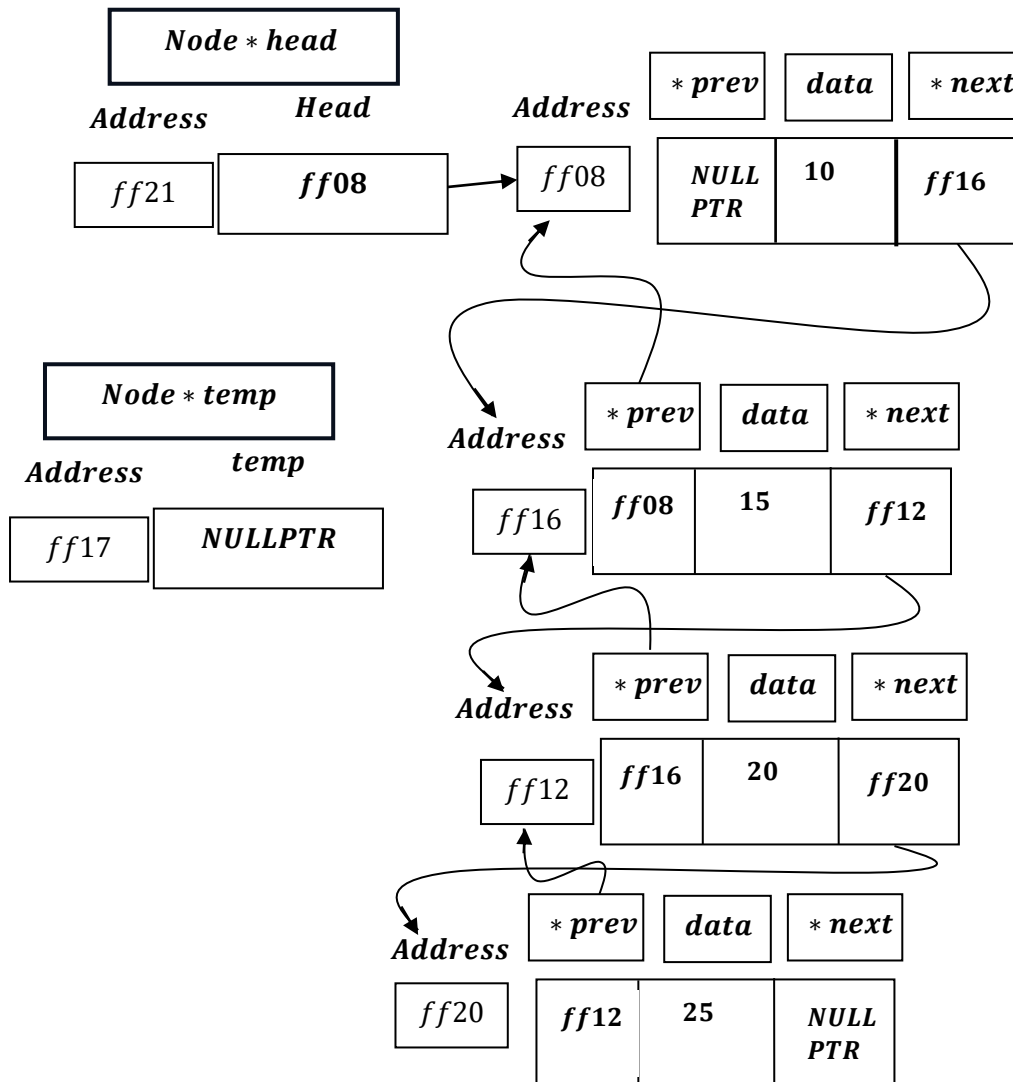
Hence , it is proved , that till `TEMP → DATA = ELEMENT` , TEMP will not be assigned to NULLPTR.

Conditions that will make TEMP pointer will point to NULLPTR :

→ HEAD points to NULLPTR.

→ ELEMENT input by USER is different and not present in any DATA section of NODES of the LIST.

Such we want to enter data : 26 after element : 24 ,but 24 is not present in the LIST.



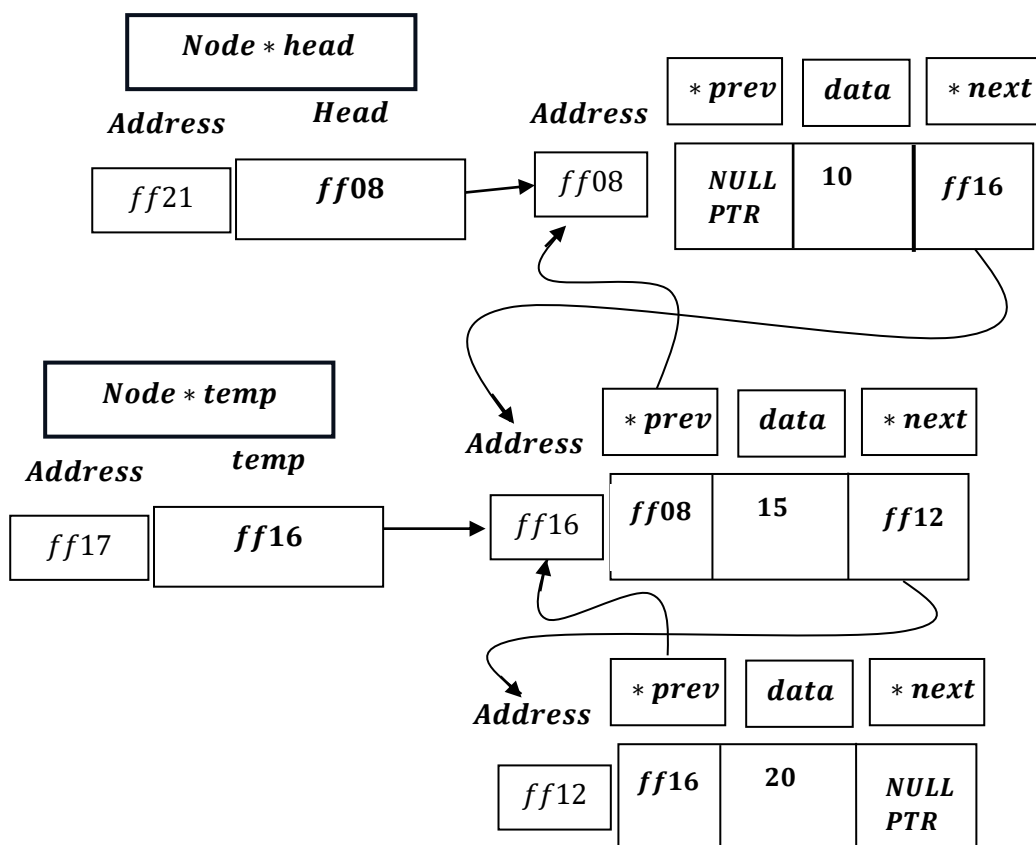
Hence,

```
if (temp == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

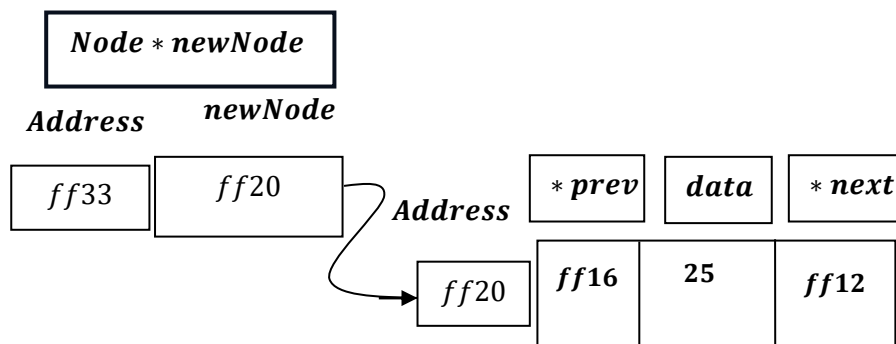
It prints output Element 24 not found in the list. And return statement ends the function or exists from the function.

```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

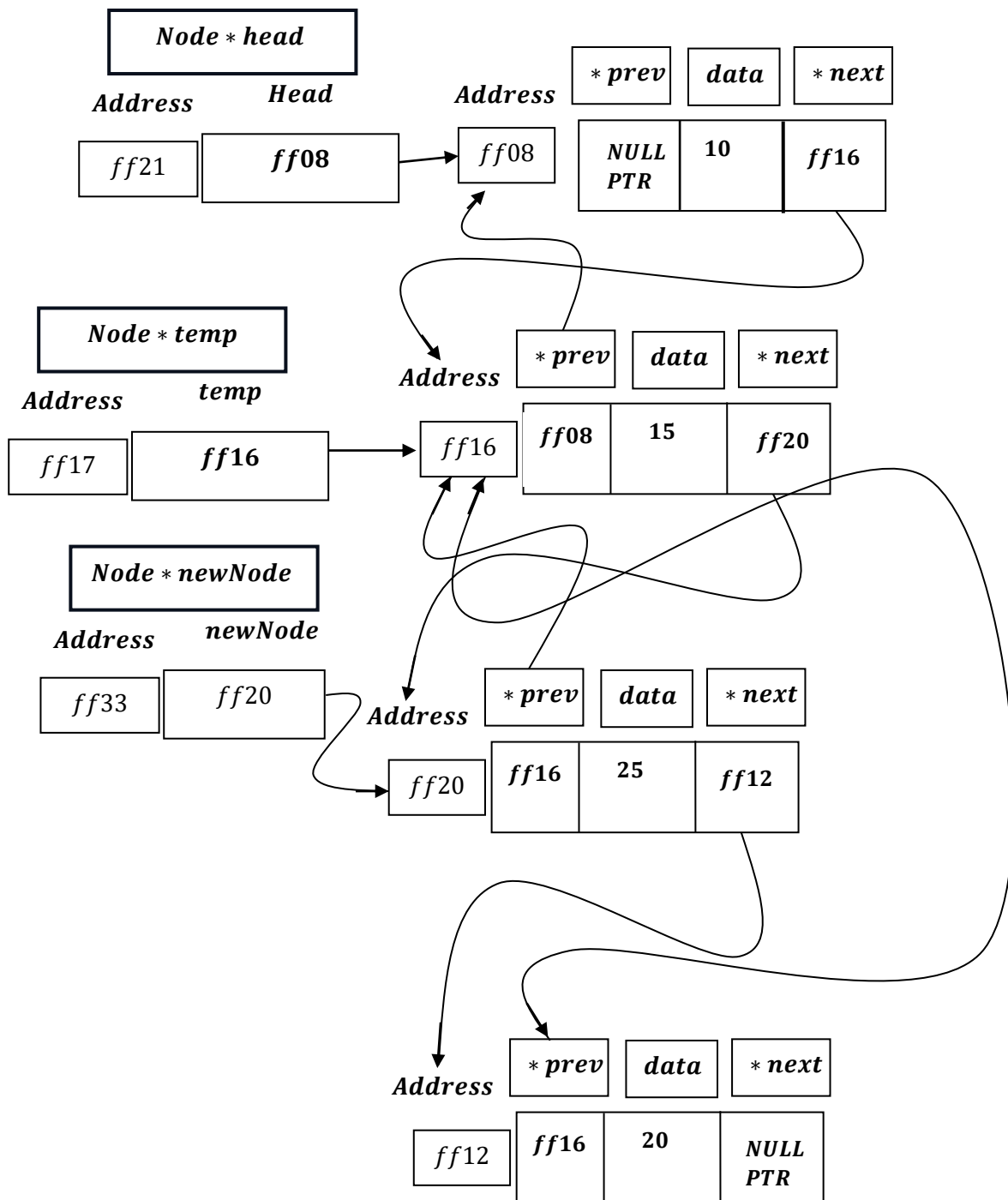
Lets we have a List:



Next we have newNode:

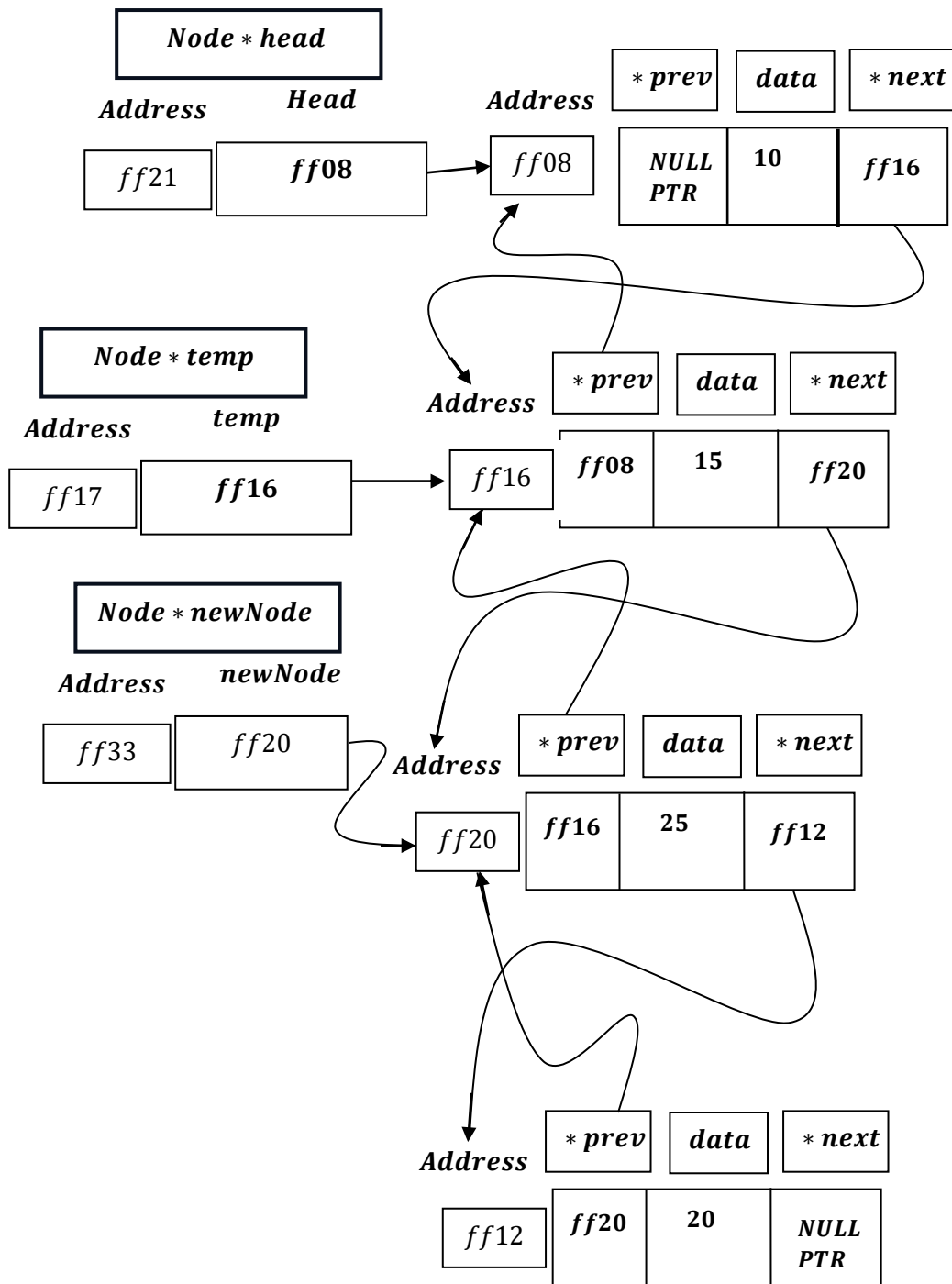


*Therefore ,adjusting newNode in the List,making $TEMP \rightarrow NEXT = newNode$
 $= ff20$.*

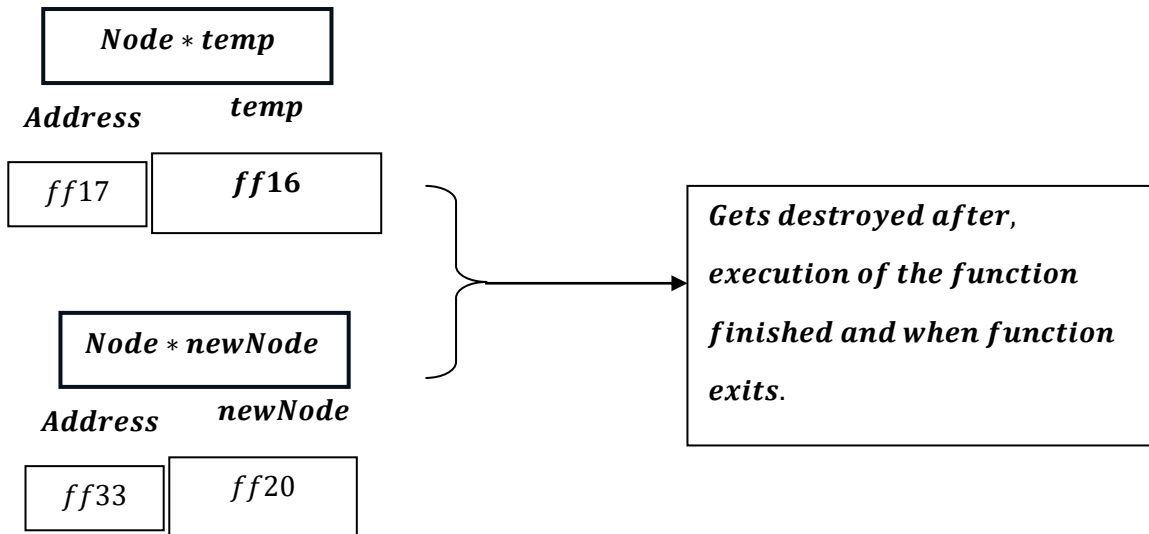


Now, we need, $\text{newNode} \rightarrow \text{next} \rightarrow \text{prev} = \text{newNode} = \text{ff20}$.

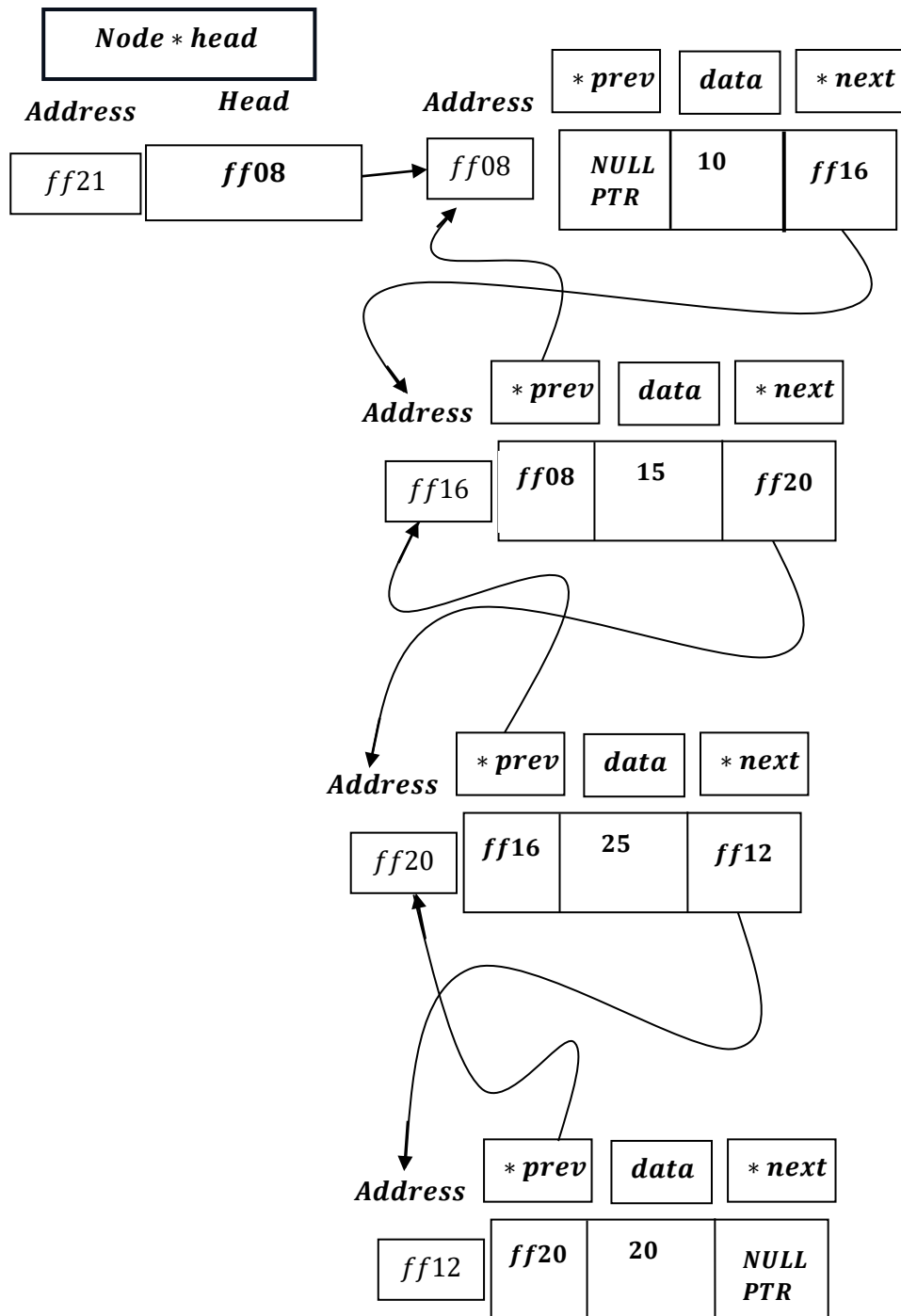
```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```



Now, **Node * newNode** , **Node * temp** temporary variables, therefore gets destroyed after function execution get finished .



Hence ,we are left with:



Doubly Linked List - Insert After Element [Working Summary]:

This function inserts a new node with a specified `data` value immediately after the first node containing a target `element` in a doubly linked list.

Working Summary

1. **Search for the Element:** The function starts at the `head` of the list and traverses it using a temporary pointer, `temp`. It continues moving `temp` to the next node until it either finds the node where `temp->data` matches the `element` or it reaches the end of the list (`temp` becomes `nullptr`).
2. **Handle "Not Found" Case:** If the loop finishes because `temp` is `nullptr`, it means the `element` was not in the list. The function prints an error message and stops.
3. **Create the New Node:** If the `element` is found, the function allocates memory for a `newNode` and sets its `data` field to the new `data` value.
4. **Perform the Insertion:** The core logic involves re-wiring the pointers to fit the `newNode` into the list. This is a four-step process:
 - o `newNode->next = temp->next;` — The new node's `next` pointer is set to point to the node that was originally after `temp`.
 - o `newNode->prev = temp;` — The new node's `prev` pointer is set to point back to `temp`.
 - o `temp->next = newNode;` — The `temp` node's `next` pointer is updated to point to the new node.
 - o `newNode->next->prev = newNode;` — The `prev` pointer of the node *following* the new node is updated to point back to the `newNode`. This step is crucial and is only done if the new node is not being inserted at the very end of the list.

After these steps, the `newNode` is correctly spliced into the list, and a success message is printed.

Time Complexity of Insert After Element Function Module

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;  $\rightarrow O(1)$ 

    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;  $\rightarrow runs\ 'n - 1'\$   

         $O(n - 1) = O(n)$  time at Worst Time Complexity
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the list." << endl;  $\rightarrow O(1)$ 
        return;  $\rightarrow O(1)$ 
    }

    Node *newNode = (Node *)malloc(sizeof(Node));  $\rightarrow O(1)$ 
    newNode->data = data;  $\rightarrow O(1)$ 
    newNode->next = temp->next;  $\rightarrow O(1)$ 
    newNode->prev = temp;  $\rightarrow O(1)$ 
    temp->next = newNode;  $\rightarrow O(1)$ 

    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;  $\rightarrow O(1)$ 
    }
    cout << data << " inserted after " << element << "." << endl;  $\rightarrow O(1)$ 
}
```

There are three case scenarios:

→ 1. Data should be inserted after the first element in the List . (Best Case)

→ 2. Position is $N + 1$, i. e. we want to insert data after n^{th} Element . (Worst Case)

→ 3. Inserted position from 2 to $N + 1$, for loop runs from 1 to $N - 1$.

While if Element is entered after first element, complexity is : $O(1)$.

And when the position is uniformly distributed from 1 to $(N + 1)$.

(Average Case i. e. Undetermined Position). That is we donot whether the position will be 1 or 2 or $N + 1$, where Node is to be inserted.

Best Case[Considered Absolute Best Case]

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr but as data to be input after the first element , hence $temp \rightarrow data = element$, therefore according to condition $temp \rightarrow data \neq element$ fails , hence loop doesn't runs. The condition verification process takes constant amount of time $O(1)$.

3. As `if condition  $temp = NULLPTR$ ` fails , hence if block doesn't execute therefore , the condition verification process takes constant amount of time $O(1)$.

4. newNode pointer of Node named structure points and allocates a memory for the Node structure , allocating for prev pointer , integer data, and next pointer , this pointing as well as allocating through `malloc` takes constant amount of time i. e. , $O(1)$.

5. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input , takes constant amount of time i.e., $O(1)$.

6. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address , which is pointed by next pointer pointed by temp pointer , the process takes constant amount of time i.e., $O(1)$.

7. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address , which is pointed by temp pointer , the process takes constant amount of time i.e., $O(1)$.

8. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address , which is pointed by newNode pointer , the process takes constant amount of time i.e., $O(1)$.

9. Now , If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` , which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e. address obtained by newNode pointer , making prev pointer pointed to address , which is being pointed by newNode. , the process takes constant amount of time i.e., $O(1)$.

10. Now we output a statement, user input data inserted at position , the process takes constant amount of time i.e., $O(1)$.

Hence , total time complexity it takes : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$, i.e. constant time complexity.

2. Best Case[When Head points to `NULLPTR`]

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr , here head points to `nullptr` , hence temp also points to `nullptr` , the loop condition fails $O(1)$.

3. As `if condition temp = NULLPTR` is true , hence if block executes and if block statements runs along with return statement exits the function , This process takes constant amount of time $O(1)$.

Hence , total time complexity it takes : $O(1) + O(1) + O(1) = O(1)$, i.e. constant time complexity.

3. When Element entered at Position $\Rightarrow N + 1$, Worst Case Scenario

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr but as data to be input after the element at n^{th} node , hence $\text{temp} \rightarrow \text{data} = \text{element}$, therefore according to condition $\text{temp} \rightarrow \text{data} \neq \text{element}$ fails , when $\text{temp} = \text{temp} \rightarrow \text{next}$, the statement inside the loop gets excuted, makes temp pointer to point at n^{th} node .

Hence the inner statement of the loop runs `n - 1` times as, it was already pointed at Head node or First node.

$$\therefore O(n - 1) = O(n).$$

3. As `if condition temp = NULLPTR` fails, hence if block doesn't execute therefore, the condition verification process takes constant amount of time $O(1)$.

4. newNode pointer of Node named structure points and allocates a memory for the Node structure, allocating for prev pointer, integer data, and next pointer, this pointing as well as allocating through `malloc` takes constant amount of time i.e., $O(1)$.

5. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input, takes constant amount of time i.e., $O(1)$.

6. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address, which is pointed by next pointer pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

7. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address, which is pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

8. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address, which is pointed by newNode pointer, the process takes constant amount of time i.e., $O(1)$.

9. Now ,If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` , which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i. e. address obtained by newNode pointer , making prev pointer pointed to address , which is being pointed by newNode. ,the process takes constant amount of time i. e., $O(1)$.

10. Now we output a statement, user input data inserted at position , the process takes constant amount of time i. e., $O(1)$.

Hence ,total time complexity it takes : $O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$, i. e. constant time complexity.

4. Inserted position from 1 to $N + 1$, while loop runs from 1 to $N - 1$.

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

→ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

→ If the Element is inserted at 3rd node, the loop's inner statement runs 2 time.

→ If the Element is inserted at 4th node, the loop's inner statement runs 3 time.

.

.

→ If the Element is inserted at $(n + 1)^{th}$ node, the loop's inner statement runs $N - 1$ time.

As when $Temp \rightarrow Data = Element$, the condition becomes false , hence no execution of loop's inner statements.

$$\therefore, \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilities then, we take the best case also i.e.

$O(1)$, when Element entered after first node = 1.

Considering ' $T(P)$ ' is time complexity of position ' P '.

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ N - 1 & , \text{when } P = 2, 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is ' N ' no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N+1}$.

Hence probability of insertion will always remain $\frac{1}{N+1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \dots + \frac{1}{N+1} \times N - 1 \right) + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1+2+3+\dots+N-1)+1\}$$

$$= \frac{1}{N+1} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{1}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{1}{N+1}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N+2}\right) + \Theta\left(\frac{1}{N+1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N+2}\right) + \Theta\left(\frac{1}{N+1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster***

than the constant 2. The dominant term is $2N$.

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i. e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant}\right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{1}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

**(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)**

**Note: –The checking that Element or Key is found or not , done in the
condition of while loop, and the condition checks `n` times .**

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probability of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probability of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding N^{th} element (p_2), if key equals to N^{th} element $= \frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the Key is searched $\times$ probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned} \therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \end{aligned}$$

$$= \frac{N+1}{2}$$

$$= \frac{N}{2} + \frac{1}{2}$$

Applying , Big Theta Θ we get :

$$= \frac{1}{2} \times (\Theta(N) + \Theta(1))$$

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert After Element</i>	<i>1. Best Case</i>	<i>$\Omega(1)$</i>
	<i>2. Worst Case</i>	<i>$O(n)$</i>
	<i>3. Average Case</i>	<i>$\Theta(n)$</i>
